Instrumenting tensor-core instructions step by step

`HMMA.16816.F16`, `HMMA.16816.F32`, `HMMA.16816.F32.BF16`

Ganesh Gopalakrishnan

SC26 tutorial — data from `TOOL_VERBOSE=1` runs of `rd_tc`

1. What the GPU runs: three HMMA flavours

`rd_tc` (2D reaction–diffusion, one step = two GEMMs) compiles to these SASS instructions (`cuobjdump -sass`):

```
fp16acc16  HMMA.16816.F16      R22, R16, R22, R8      // D, A, B, C
fp16acc32  HMMA.16816.F32      R16, R8.reuse, R22, R16
bf16acc32  HMMA.16816.F32.BF16 R16, R8.reuse, R22, R16
```

- Operand order is always **D**, **A**, **B**, **C**: $D = A \times B + C$ for one $16 \times 8 \times 16$ tile per warp.
- Each kernel holds 16 HMMAs (verified: 16 "Instrumenting instruction HMMA" lines per run).
- The first HMMA of every chain has `RZ` as its C operand: the accumulator starts at zero.
- NixNan sees only SASS text and operand structure — it does not know about `mma_sync`.

2. The table that describes an HMMA: `instructions.json`

The opcode string (with modifiers) is the key; each operand slot gets a {type, count}:

	D (op 0)	A (op 1)	B (op 2)	C (op 3)
`HMMA.16816.F16`	f16 ×4	f16 ×8	f16 ×4	f16 ×4
`HMMA.16816.F32`	f32 ×4	f16 ×8	f16 ×4	f32 ×4
`HMMA.16816.F32.BF16`	f32 ×4	bf16 ×8	bf16 ×4	f32 ×4

- `count` = number of *values*, not registers. f16/bf16 are packed two per 32-bit register.
- `get_num_regs`: f16/bf16 $\to \lceil c/2 \rceil$, f32 $\to c$, f64 $\to 2c$.
- Hence F16: D=2, A=4, B=2, C=2 registers. F32 and F32.BF16: D=4, A=4, B=2, C=4.
- An opcode with no table entry (e.g. `F2FP.PACK`) is skipped: nothing is inserted.

3. Verbose log: what NixNan decided (first launch only)

```
Instrumenting instruction HMMA.16816.F16 R22, R16, R22, R8 ;
Instrumenting instruction with 3 registers          // AFTER call
Instrumenting operand 0. div0: 0, regs: 2, count: 4
Instrumenting instruction with 11 registers         // BEFORE call
Instrumenting operand 1. div0: 0, regs: 4
Instrumenting operand 2. div0: 0, regs: 2
Instrumenting operand 3. div0: 0, regs: 2
```

	AFTER (op 0)	BEFORE (ops 1–3)	log lines identical for
`HMMA.16816.F16`	3	11	all 16 HMMAs
`HMMA.16816.F32`	5	13	all 16 HMMAs
`HMMA.16816.F32.BF16`	5	13	all 16 HMMAs

The number is the device function's `arg_count`: AFTER $= 1 + \mathrm{regs}(D)$; BEFORE $= 3$ reginfo words $+ \mathrm{regs}(A, B, C)$.

F16: $1 + 2 = 3$, $3 + 4 + 2 + 2 = 11$. F32: $1 + 4 = 5$, $3 + 4 + 2 + 4 = 13$.

4. Compact encoding: one 32-bit `reginfo` per operand

Bitfield (real struct from `reginfo.cuh`): count:6, type:4, half_h0, half_h1, div0, operand:2, num_regs:4, f64high, reserved. Type codes: f16=0, bf16=1, f32=2, f64=3. Words below were printed by a g++ program using that struct, filled in as NixNan fills it (not dumped from the running tool).

	D	A	B	C
F16	0x00010C04	0x00022C08	0x00014C04	0x00016C04
F32	0x00020C84	0x00022C08	0x00014C04	0x00026C84
F32.BF16	0x00020C84	0x00022C48	0x00014C44	0x00026C84

Decoding 0x00022C48 (A of F32.BF16): count=8, type=1 (bf16), h0=h1=1, operand=1, num_regs=4.

Why compact: the words travel as *call arguments* on the GPU, one per operand, so the device function can classify with no table lookup.

5. Which physical registers are picked up

`get_regs` turns each `REG` operand into `num_regs` consecutive `nvbit_add_call_arg_reg_val` calls (Rn, Rn+1, ...).

instruction	D (AFTER)	A	B	C (BEFORE)
`F16 R22,R16,R22,R8`	R22–R23	R16–R19	R22–R23	R8–R9
`F32 R16,R8,R22,R16`	R16–R19	R8–R11	R22–R23	R16–R19
`BF16 R16,R8,R22,R16`	R16–R19	R8–R11	R22–R23	R16–R19

- **Why two call sites:** D overlaps B (R22) and, for F32, C (R16). `IPOINT_BEFORE` reads the inputs before HMMA overwrites them; `IPOINT_AFTER` reads the result D.
- First HMMA in each chain has C=RZ: NixNan sends `RZ` (index clamped to 255) in every C slot; it reads as 0 (inferred from the code, not separately tested).
- `.reuse` on A is an operand-cache hint; it does not change the register read.

6. On the GPU: `nixnan_check_regs` classifies the fragments

Every thread of the warp passes its own fragment registers (a tile is spread over 32 lanes).

1. Walk the args: read a `reginfo`, then `num_regs` values with `va_arg`.
2. Classify per type: f16 → `half2_classify` (two halves per register), bf16 → `bf162_classify`, f32 → `float_classify`. Each yields NaN / Inf / −Inf / subnormal bits.
3. OR into `exces[operand]`, then shuffle-OR across the warp (the 4 operand slots are why `OPERANDS=4`).
4. Lowest active lane only: `recorder.record(inst_id, flags, op)` does an `atomicAdd` on a counter indexed by `id«7 | op«5 | flags`.
5. If the old count was 0 (first time this instruction/operand/flag-set is seen), push an `exception_info` to the channel. Repeats are only counted.

This is why the report says "detected in operand k of instruction … of type f16": the operand slot and the type from the table travel with the record.

7. Result: what the instrumented HMMAs reported

```
[infinity] operand 0  HMMA.16816.F16 R26, R4, R30, R26   f16  // born in D
[infinity] operand 3  HMMA.16816.F16 R4, R16.reuse, R8, R4    // reused as C
[NaN]      operand 0  HMMA.16816.F16 R22, R16, R22, R8        // 0 x inf
```

- fp16acc16: the Inf appears in operand 0 (D) of `HMMA.16816.F16` at step 230; NaN follows at step 231 (a single Inf spreads along a whole row and column of U' because P is dense).
- fp16acc32 / bf16acc32: the Inf is only seen as an *input* (operands 1–3) of the next `HMMA.16816.F32[.BF16]`: it is created by `F2FP.PACK`, which has no table entry.
- Not visible: tile-internal products. NixNan inspects the register file around the instruction, not the inside of the tensor core.